

1. Inverted Index & TF-IDF

Structure: term $\rightarrow$ [(docID, tf), (docID, tf), ...] with df per term

$$w(t, d) = tf \times \log_2\left(\frac{N}{df}\right) \quad [\text{on cribsheet} - \log \text{ table is } \log_{10}]$$

WHY: IDF penalises common terms and rewards rare discriminating terms

(!) EXAM TRAPS:

- If df = N $\rightarrow$ IDF = $\log(1) = 0 \rightarrow$ term scores ZERO everywhere (e.g. "interest")
- Count tf only AFTER stopwords removal -- recount carefully from scratch
- Equal scores = tied rank (state: "Doc1 = Doc3 > Doc2")

2. Vector Space Model - Cosine Similarity

$$\cos(D, Q) = \frac{D \cdot Q}{|D| \times |Q|} \quad \text{where } |v| = \sqrt{\sum v_i^2} \quad [\text{on cribsheet}]$$

Steps:

- 1. Dot product: sum $d_i \cdot q_i$ for terms present in BOTH doc and query
- 2. $|D|$ = sqrt(sum of ALL term weight squares in doc)
- 3. $|Q|$ = sqrt(sum of query term weight squares)
- Shorter, focused docs score higher -- cosine normalises by vector length

3. Evaluation Metrics

$$P = \frac{|Rel \cap Ret|}{|Ret|} \quad R = \frac{|Rel \cap Ret|}{|Rel|} \quad F_1 = \frac{2PR}{P+R}$$

$$AP = \frac{1}{|R_{total}|} \sum_k P(k) \cdot rel(k) \quad [\text{divide by TOTAL relevant, not retrieved}]$$

$$DCG_p = rel_1 + \sum_{i=2}^p \frac{rel_i}{\log_2(i+1)} \quad nDCG = \frac{DCG}{IDCG} \quad [\log_2 \text{ values given in Q}]$$

(!) CRITICAL TRAPS:

- AP: always divide by TOTAL relevant -- un-retrieved docs contribute 0 but per
- R-P graph: rank_k = k / P(at that recall point) -- derive all ranks this way
- Interpolated P(r) = MAX precision at any recall $\geq r$ (look AHEAD in graph)
- IDCG = ideal DCG -- sort relevance grades highest first

4. Heap's Law & Zipf's Law

$$V = K \cdot n^\beta \quad (\beta \approx 0.5) \Rightarrow V = K\sqrt{n} \quad \text{where } n = \text{total tokens}$$

$$r \times P(w_r | C) = A \Rightarrow P(w_1) = A, \quad P(w_2) = \frac{A}{2}, \quad \dots$$

- Heap: n = docs x avgdl. Find $K = V/\sqrt{n}$. $V_{\text{new}} = K \cdot \sqrt{n_{\text{new}}}$
- Sublinear growth: larger collections gain fewer NEW terms per doc added
- Zipf: A = given_rank x given_freq/T. Top 4 words cover ~20.8% of all tokens

5. Rocchio Relevance Feedback

$$\vec{q}_{\text{new}} = \alpha \vec{q}_0 + \frac{\beta}{|D_R|} \sum \vec{d} - \frac{\gamma}{|D_N|} \sum \vec{d} \quad [\text{on cribsheet}]$$

Typical: $\alpha=1$ $\beta=0.75$ $\gamma=0.25$ Set NEGATIVE weights to 0

Steps:

- 1. q_{orig} = raw TF of query string (e.g. bargain=3, books=2)
- 2. Relevant centroid = average of relevant doc vectors (divide by $|D_R|$)
- 3. Non-rel centroid = average of non-relevant doc vectors (divide by $|D_N|$)
- 4. Apply formula term by term -- drop any negatives (set to 0)

"Find Similar": $\alpha=0$, $\beta=1$, $\gamma=0$ (clicked doc becomes the new query)

6. Language Models - JM Smoothing

$$P(w|d) = (1 - \lambda) \frac{tf_{w,d}}{|d|} + \lambda \cdot P(w|C) \quad [\text{on cribsheet}]$$

$$\text{Score}(q|d) = \prod_{w \in q} P(w|d) \quad \text{multiply across all query terms}$$

WHY: Zero-probability fix -- missing term ($tf=0$) still gets $L \cdot P(w|C) > 0$

- Rare terms (low $P(w|C)$) discriminate MORE -- high tf of rare term wins ranking
- $L=0.5$: equal weight to document-specific and collection-level probability

(!) Doc with higher tf of RARER term ranks above doc with higher tf of com

7. BIM -- Binary Independence Model

$$RSV = \sum_{i: d_i=1} \log \frac{p_i(1-s_i)}{s_i(1-p_i)} \quad [\text{sum ONLY where } d_i=1, \text{ i.e. term present in doc}]$$

No info: $p_i = 0.5$ $s_i = df / N$

$$p_i = \frac{r_i + 0.5}{R + 1} \quad s_i = \frac{df_i - r_i + 0.5}{N - R + 1} \quad (\text{with relevance feedback})$$

(!) TRAPS:

- Only sum for terms where $d_i=1$ (term IS present in doc AND in query)
- Relevant doc WITHOUT query term $\rightarrow RSV=0$ (major BIM limitation)
- Non-relevant doc WITH term scores same as relevant -- BIM cannot distinguish
- Use log base 10 -- look up values from the cribsheet log table

8. IR Model Assumptions - Always 4 marks!

- 1. Term independence (BoW): "New York" != "New"+"York" -- context is lost
- 2. Document independence: ignores diversity -- 10 identical docs is useless
- 3. Static relevance: ignores time -- 2019 COVID doc misleading in 2026
- 4. Topical = User relevance: ignores readability, credibility, authority

For EACH: give the assumption + a concrete real-world example = full marks

9. HITS Algorithm - Hub & Authority

Hub: links TO many good authorities **Authority:** linked BY many good hubs

$$a(p) = \sum_{q \rightarrow p} h(q) \quad h(p) = \sum_{p \rightarrow q} a(q) \quad [\text{iterate, normalise each step}]$$

Matrix method (exam: "using matrices, do NOT resolve equations"):

1. Build A: $A[i][j]=1$ if page i links TO page j
2. Compute transpose A^T
3. $Ma = A^T * A \rightarrow$ write eigenvalue eqn: $Ma * a = \lambda * a$ (authority)
4. $Mh = A * A^T \rightarrow$ write eigenvalue eqn: $Mh * h = \lambda * h$ (hub)
5. Diagonal $Ma[i][i]$ = in-degree of page i <-- verify matrix with this!

Iterative: start $h=(1,1,...)$. Update ALL a then ALL h . Normalise by max each step

10. TAAT vs DAAT Index Traversal

TAAT: Process one FULL posting list then next. Memory: $O(N)$ -- bad!

DAAT: Process all terms for ONE doc then move on. Memory: $O(K)$ -- good!

MaxScore / WAND (dynamic pruning):

- Precompute $UB(t)$ = max score contribution of term t to any doc in its list
- Skip doc if $\text{partial_score} + \text{remaining UBs} < \text{current threshold}$ -- saves CPU

11. Index Compression - Lossy vs Lossless

Lossless: No info lost. d-gaps, VByte, Gamma codes, PForDelta

-> Measure: compression ratio + decompression speed. MAP unchanged.

Lossy: Info discarded. Static pruning, stopword removal, quantisation

-> Measure: compression ratio AND MAP/nDCG before vs after

- Acceptable if MAP drops < 1-2% with significant index size reduction

12. Exam Strategy - 1h 30min, 60 marks

- Q1 (~25 min, ~20 marks): Inverted index -> TF-IDF -> Cosine -> P/R/AP/nDCG
- Q2 (~25 min, ~20 marks): Heap's -> Rocchio -> JM Smoothing -> Conceptual
- Q3 (~25 min, ~20 marks): Neural IR -> PyTerrier -> 4 Assumptions or HITS

(!) Write: formula -> substitute values -> compute -> state result = partial marks

- Check: IDF=0 trap, AP/total_relevant, normalise HITS each iteration

13. Neural IR - Architecture Trade-offs

Bi-Encoder: Encodes q and d SEPARATELY. Pre-index docs offline -> fast AND

- Use as FIRST STAGE -- finds semantic matches (handles vocabulary mismatch)

Cross-Encoder: Encodes (q,d) JOINTLY -- full interaction, best quality but slow

- Cannot pre-compute -> use as RE-RANKER on top-100 only, never first stage

ColBERT: Token-level late interaction -- best of both (pre-encode doc tokens)

$$\text{MaxSim}(Q, D) = \sum_{q_i \in Q} \max_{d_j \in D} \text{sim}(q_i, d_j) \quad [\text{on cribsheet}]$$

(!) NaN p-value: re-ranker has same 1000 candidates -> Recall@1000 identical

14. PyTerrier Pipelines - Up to 16 marks!

> > Then %n Top-n cutoff | Union * * Feature-Union [on cribsheet]

PRF pipeline: `bm25 % 3 >> rewriter1(index) >> bm25`

- `rewriter1`: sees top-3 DOCS + index. Uses Bo1 / RM3 (pseudo-relevance feedback)
- `rewriter2 >> bm25`: sees ONLY query string. Uses thesaurus / Word2Vec

Experiment template:

```
pt.Experiment([baseline, system], topics, qrels,
              eval_metrics=["ndcg_cut_10", "map", "recall_1000"], baseline=0)
```

- `baseline=0` enables auto paired t-test. NDCG@10 = top-10 user quality

15. Dense Retrieval + Passage Aggregation

(!) **PROBLEM: BERT 512-token limit -- single vector loses long-document context**

FIX: Split into overlapping passages, encode each one independently

MaxPassage: score = BEST single passage (one strong section proves relevance)

SumPassage: score = SUM of all passage scores (evidence spread throughout doc)

- Relevant content SPREAD through doc? -> Use SumPassage (accumulates evidence)
- ONE highly relevant section? -> MaxPassage is fine